

AI Panorama Journal

First I downloaded all the images into my ai-panorama directory and ran the current [panorama.py](#) code that I have. The result was 3 images stitched together, and the rest of the 5 got left out.

Image :



Then with [Claude.ai](#) help, I asked for this prompt. Why doesn't all of the 8 images stitch together only 3? And what would I have to do to make sure all of the eight images stitch together.

- When you give it all 8 images at once, it matches every pair and scores how confident each match is. Any image whose best match falls below the confidence threshold (default 1.0) gets dropped. Then the stitcher keeps only the largest group of images that are still connected to each other. So only the 3 strongest neighbors survive, and it gives you those without any error.

Then I added the image from canvas with all 8 pictures stitched together, and asked how was it possible.

- It stitches neighbors in pairs instead of all at once. Then it stitches those results together, and repeats until one panorama is left. It loosens the settings only when a pair fails. It raises `setRegistrationResol` from 0.6 to 0.8, so more feature points get detected in each image. It saves every intermediate merge in a `steps/` folder, so you can see which pair failed if something breaks.

I Asked what could I improve in my code to make it work like the particular order and hierarchical.

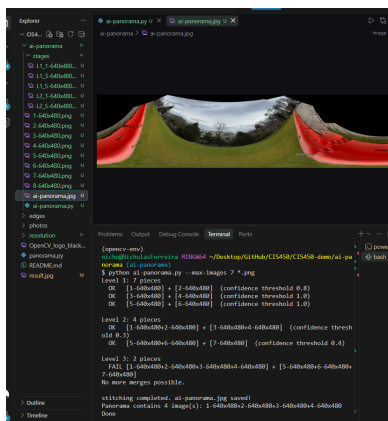
- The single stitch is replaced by `hierarchical_stitch()`. Instead of one `stitcher.stitch(imgs)` call, it stitches neighbor pairs (1+2, 3+4, ...) and then merges those results in pairs until one panorama is left.
- `try_stitch()` It starts at the default confidence threshold (1.0). Only when a pair fails does it lower the threshold, trying 0.6, 0.3, and 0.1, and then the other mode.

- Images are resized to at most 1200 px wide. But Since the Images are already 640x480, they will remain the same.
- Registration resolution is raised from 0.6 to 0.8. You can change that with `--resol`.
- Every successful merge is saved as `step_L1_1_2.jpg`, `step_L2_1_2_3_4.jpg`, and so on, so you can see where it works and where it fails.

When I ran the new code, the first level worked. Pairs 1+2, 3+4 and 5+6 all stitched. Then the program crashed with an error: `OpenCL error CL_OUT_OF_RESOURCES`. I asked Claude what it meant.

- OpenCV uses OpenCL to run on the graphics card, and the GPU ran out of memory when the merged images got bigger.
- The fix was adding `cv.ocl.setUseOpenCL(False)` at the top so it runs on the CPU instead.

After the fix, the program said it stitched 4 images, but the picture was just grass and the orange barrier bent into a bowl shape.

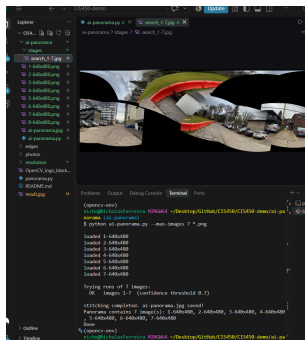


- Images 1–4 only merged at confidence 0.3. That low, the stitcher accepts wrong matches and warps the picture to fit them.
- Stitching panoramas together doesn't work well, because a panorama is already curved and the stitcher expects normal photos.

Next Claude changed the code so it always stitches the original photos. It started from the best pair and added one photo at a time.

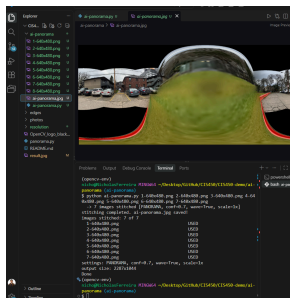
- Every neighbouring pair (1+2 up to 6+7) stitched, so all 7 images can connect.
- But it added image 1 first, then couldn't add image 4 and got stuck with only 3 images.

Then Claude changed it to try every group of neighbouring images, longest first. It also checked `stitcher.component()` to make sure every image was really used. This time it said all 7 images stitched, but the picture was scrambled, with pieces sideways and in the wrong places.



- `cv.Stitcher` compares every image with every other image. This parking lot has lots of repeated things like cars, windows and orange barriers, so it matched images that don't actually overlap.
- I learned that the terminal saying "success" doesn't mean the panorama is right. I have to look at the picture.

I also tried two other versions of the code. One used a flat homography starting from image 1, and it failed on the first pair with "homography sends corners too far away". A flat picture can't hold a scene that turns that far. The other version tried lots of settings and picked the result with the most images. It said "7 of 7 USED", but the picture had a giant blurry grass bubble in the middle.



Finally, Claude added a "detailed" engine that does each step of stitching separately, based on OpenCV's `stitching_detailed.py` sample.

- It finds SIFT features in each image.
- It matches each image only with its neighbour, so a car in image 1 can't be matched to a car in image 6.
- It estimates how the camera turned between photos and warps every image onto a sphere.
- It balances the brightness and blends the seams.

I ran `python ai-panorama.py --max-images 7 *.png`, and images 1–7 stitched at confidence 0.5 in the right order: barrier and grass, parking lot, brick building, tall grey building, and the garage. Image 8 was left out, since it's too different from image 7.



What I learned:

- Lowering the confidence threshold lets more images in, but too low lets bad matches in too.
- Matching only neighbouring images stops false matches between photos that don't overlap.
- "Success" or "7 of 7 used" doesn't mean the result is correct. You have to look at it.
- The AI gave me several answers that didn't work, and testing each one is how I found the one that did.